

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Российский экономический университет имени Г.В. Плеханова»
Пермский институт (филиал)

РАЗРАБОТКА ПРИЛОЖЕНИЯ «МЕНЕДЖЕР ЗАДАЧ»
С ИСПОЛЬЗОВАНИЕМ WPF, LINQ И БИБЛИОТЕК КЛАССОВ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к практической работе

Выполнил: студент группы: ИПо-32
Карабатов Дмитрий Андреевич

г.Пермь 2026г

Оглавление

ВВЕДЕНИЕ	3
1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	4
1. Основные требования:	4
2. Дополнительные задания:	4
2. ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ	5
2.1 Диаграмма классов	5
2.2 Описание классов	5
3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	6
3.1 Создание решения и проектов	6
3.2 Реализация библиотеки классов TaskManager.Core	6
3.3 Реализация WPF-интерфейса	10
4. ТЕСТИРОВАНИЕ	12
5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	13
ЗАКЛЮЧЕНИЕ	14
СПИСОК ЛИТЕРАТУРЫ	15

ВВЕДЕНИЕ

Целью данной работы является разработка приложения для управления задачами на языке C# с использованием технологии WPF, библиотеки LINQ для работы с коллекциями и принципов объектно-ориентированного программирования.

Приложение позволяет:

- добавлять, редактировать и удалять задачи;
- фильтровать задачи по статусу;
- выполнять поиск по названию или описанию;
- сортировать задачи по приоритету и сроку выполнения;
- сохранять и загружать список задач в формате JSON;
- просматривать статистику (общее количество, завершённые, просроченные).

Программа состоит из трёх проектов:

- TaskManager.Core — библиотека классов с бизнес-логикой;
- TaskManager.Wpf — пользовательский интерфейс на WPF;
- TaskManager.Tests — модульные тесты на базе xUnit.

Разработка выполнена в среде Microsoft Visual Studio, язык C#, целевая платформа .NET.

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Цель: Практика работы с классами, коллекциями, файловым вводом-выводом, LINQ и основами ООП в C#.

1. Основные требования:

- 1.1 Добавление новой задачи (название, описание, приоритет, срок выполнения, статус).
- 1.2 Просмотр списка всех задач.
- 1.3 Фильтрация задач по статусу.
- 1.4 Поиск задач по названию или описанию.
- 1.5 Редактирование существующих задач.
- 1.6 Удаление задач.
- 1.7 Сохранение и загрузка задач в/из файла JSON.
- 1.8 Основная логика в динамической библиотеке, интерфейс — WPF.
- 1.9 Модульные тесты для классов библиотеки.

2. Дополнительные задания:

- 2.1 Сортировка задач по приоритету или сроку выполнения.
- 2.2 Пометка задачи как "Важная".
- 2.3 Статистика (завершено, просрочено).

2. ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ

2.1 Диаграмма классов

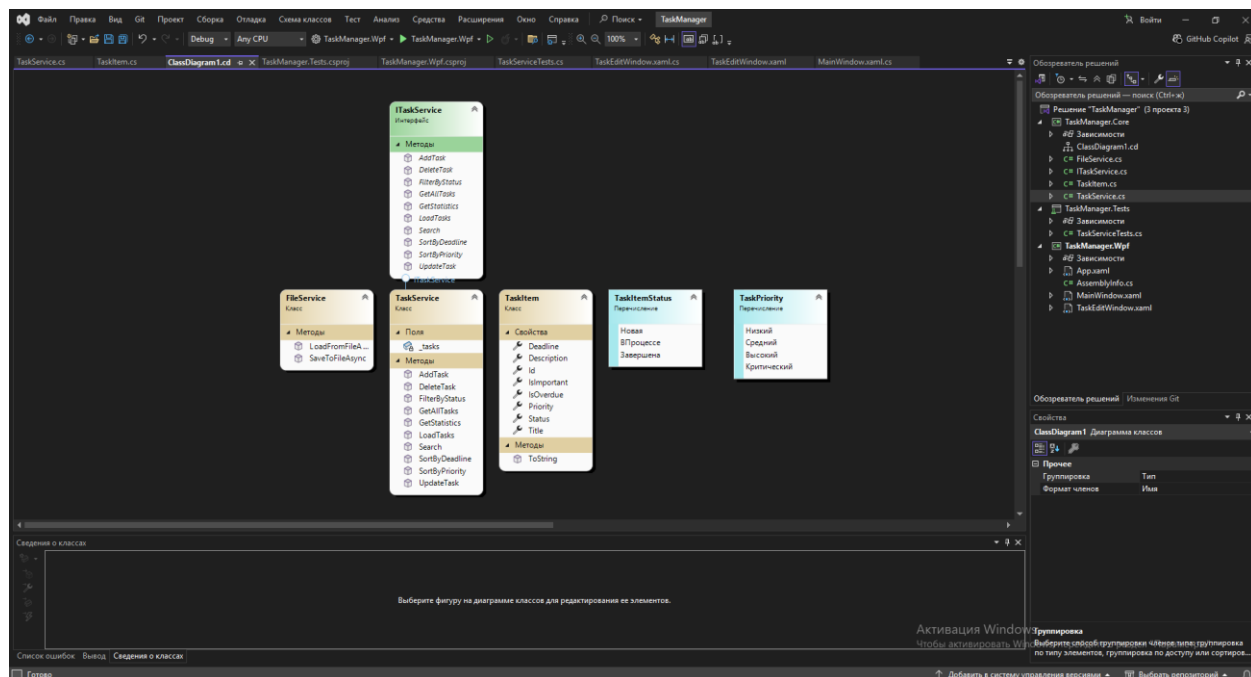


Рисунок 1 — Диаграмма классов приложения

2.2 Описание классов

Класс `TaskItem` — модель данных, представляющая одну задачу. Содержит свойства: `Id`, `Title`, `Description`, `Priority`, `Deadline`, `Status`, `IsImportant`, `IsOverdue`.

Перечисление `TaskItemStatus` определяет возможные статусы задачи: Новая, ВПроцессе, Завершена.

Перечисление `TaskPriority` определяет уровни приоритета: Низкий, Средний, Высокий, Критический.

Интерфейс `ITaskService` определяет контракт для работы с задачами: добавление, удаление, редактирование, фильтрация, поиск, сортировка, получение статистики.

Класс `TaskService` реализует интерфейс `ITaskService`. Хранит задачи в коллекции `List<TaskItem>`. Для фильтрации и поиска использует LINQ-запросы (`Where`, `OrderBy`, `Count`, `FirstOrDefault`).

Класс `FileService` отвечает за сохранение и загрузку задач в формате JSON с использованием `System.Text.Json`.

3. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

3.1 Создание решения и проектов

Решение TaskManager создано в Visual Studio. В него добавлены три проекта: библиотека классов TaskManager.Core, WPF-приложение TaskManager.Wpf, тестовый проект TaskManager.Tests.

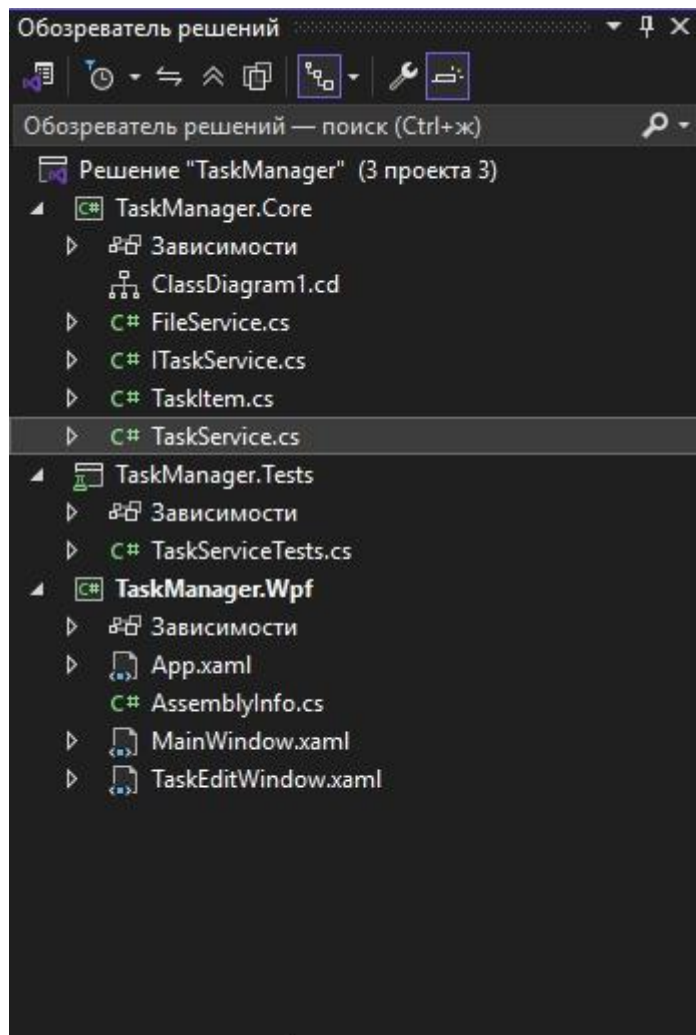
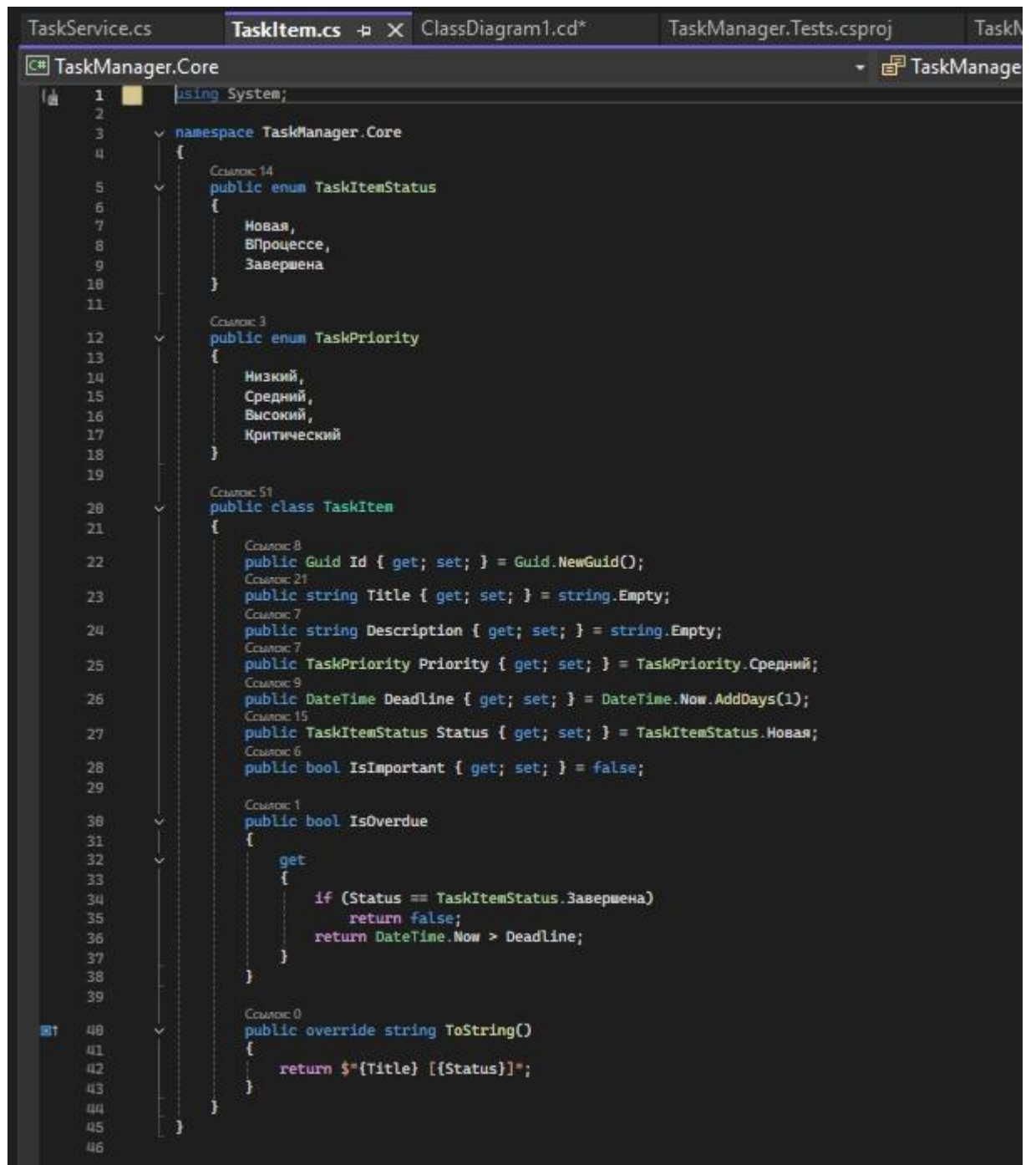


Рисунок 2 — Структура решения в обозревателе решений

3.2 Реализация библиотеки классов TaskManager.Core

Основная бизнес-логика вынесена в отдельную библиотеку классов. Это позволяет отделить логику от интерфейса и упрощает тестирование.

Класс TaskItem — модель задачи:



```
TaskService.cs TaskItem.cs X ClassDiagram1.cd* TaskManager.Tests.csproj TaskM
[C#] TaskManager.Core
1 using System;
2
3 namespace TaskManager.Core
4 {
5     Ссылка: 14
6     public enum TaskItemStatus
7     {
8         Новая,
9         ВПроцессе,
10        Завершена
11    }
12
13    Ссылка: 3
14    public enum TaskPriority
15    {
16        Низкий,
17        Средний,
18        Высокий,
19        Критический
20    }
21
22    Ссылка: 51
23    public class TaskItem
24    {
25        Ссылка: 8
26        public Guid Id { get; set; } = Guid.NewGuid();
27        Ссылка: 21
28        public string Title { get; set; } = string.Empty;
29        Ссылка: 7
30        public string Description { get; set; } = string.Empty;
31        Ссылка: 7
32        public TaskPriority Priority { get; set; } = TaskPriority.Средний;
33        Ссылка: 9
34        public DateTime Deadline { get; set; } = DateTime.Now.AddDays(1);
35        Ссылка: 15
36        public TaskItemStatus Status { get; set; } = TaskItemStatus.Новая;
37        Ссылка: 6
38        public bool IsImportant { get; set; } = false;
39
40        Ссылка: 1
41        public bool IsOverdue
42        {
43            get
44            {
45                if (Status == TaskItemStatus.Завершена)
46                    return false;
47                return DateTime.Now > Deadline;
48            }
49        }
50
51        Ссылка: 0
52        public override string ToString()
53        {
54            return $"{Title} [{Status}]";
55        }
56    }
57 }
```

Рисунок 3 — Код класса TaskItem

Класс TaskService содержит всю логику работы с задачами:

```
TaskService.cs TaskItem.cs ClassDiagram1.cd* TaskManager.Tests.csproj TaskManager.Wpf.csproj
TaskManager.Core
TaskManager.Core.TaskService
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4
5 namespace TaskManager.Core
6 {
7     public class TaskService : ITaskService
8     {
9         private List<TaskItem> _tasks = new List<TaskItem>();
10
11         public TaskItem AddTask(TaskItem task)
12         {
13             if (task == null)
14                 throw new ArgumentNullException(nameof(task));
15
16             if (string.IsNullOrEmpty(task.Title))
17                 throw new ArgumentException("Название задачи не может быть пустым");
18
19             task.Id = Guid.NewGuid();
20             _tasks.Add(task);
21             return task;
22         }
23
24         public List<TaskItem> GetAllTasks()
25         {
26             return new List<TaskItem>(_tasks);
27         }
28
29         public List<TaskItem> FilterByStatus(TaskItemStatus status)
30         {
31             return _tasks.Where(t => t.Status == status).ToList();
32         }
33
34         public List<TaskItem> Search(string query)
35         {
36             if (string.IsNullOrEmpty(query))
37                 return GetAllTasks();
38
39             string lowerQuery = query.ToLower();
40
41             return _tasks.Where(t =>
42                 t.Title.ToLower().Contains(lowerQuery) ||
43                 t.Description.ToLower().Contains(lowerQuery)
44             ).ToList();
45         }
46
47         public bool UpdateTask(TaskItem updatedTask)
48         {
49             TaskItem existing = _tasks.FirstOrDefault(t => t.Id == updatedTask.Id);
50
51             if (existing == null)
52                 return false;
53         }
54     }
55 }
```

Рисунок 4 — Код класса TaskService

```
TaskService.cs TaskItem.cs ClassDiagram1.cd* TaskManager.Tests.csproj TaskManager.Core
TaskManager.Core
50
51     if (existing == null)
52         return false;
53
54     existing.Title = updatedTask.Title;
55     existing.Description = updatedTask.Description;
56     existing.Priority = updatedTask.Priority;
57     existing.Deadline = updatedTask.Deadline;
58     existing.Status = updatedTask.Status;
59     existing.IsImportant = updatedTask.IsImportant;
60
61     return true;
62 }
63
64     COMMIT 3
65     public bool DeleteTask(Guid id)
66     {
67         TaskItem task = _tasks.FirstOrDefault(t => t.Id == id);
68
69         if (task == null)
70             return false;
71
72         _tasks.Remove(task);
73         return true;
74     }
75
76     COMMIT 1
77     public List<TaskItem> SortByPriority()
78     {
79         return _tasks.OrderByDescending(t => t.Priority).ToList();
80     }
81
82     COMMIT 1
83     public List<TaskItem> SortByDeadline()
84     {
85         return _tasks.OrderBy(t => t.Deadline).ToList();
86     }
87
88     COMMIT 3
89     public (int total, int completed, int overdue) GetStatistics()
90     {
91         int total = _tasks.Count;
92         int completed = _tasks.Count(t => t.Status == TaskItemStatus.Завершена);
93         int overdue = _tasks.Count(t => t.IsOverdue);
94
95         return (total, completed, overdue);
96     }
97
98     COMMIT 2
99     public void LoadTasks(List<TaskItem> tasks)
100     {
101         _tasks.Clear();
102         if (tasks != null)
103             _tasks.AddRange(tasks);
104     }
105 }
```

Рисунок 4.1 — Код класса TaskService

Класс FileService реализует сохранение и загрузку в JSON:

```
1 using System;
2 using System.Collections.Generic;
3 using System.IO;
4 using System.Text.Json;
5 using System.Threading.Tasks;
6
7 namespace TaskManager.Core
8 {
9     public class FileService
10     {
11         public async Task SaveToFileAsync(string filePath, List<TaskItem> tasks)
12         {
13             JsonSerializerOptions options = new JsonSerializerOptions
14             {
15                 WriteIndented = true
16             };
17
18             string json = JsonSerializer.Serialize(tasks, options);
19             await File.WriteAllTextAsync(filePath, json);
20         }
21
22         public async Task<List<TaskItem>> LoadFromFileAsync(string filePath)
23         {
24             if (!File.Exists(filePath))
25                 return new List<TaskItem>();
26
27             string json = await File.ReadAllTextAsync(filePath);
28
29             List<TaskItem> tasks = JsonSerializer.Deserialize<List<TaskItem>>(json);
30
31             return tasks ?? new List<TaskItem>();
32         }
33     }
34 }
35
```

Рисунок 5 — Код класса FileService

3.3 Реализация WPF-интерфейса

Пользовательский интерфейс реализован на технологии WPF. Главное окно содержит таблицу задач, кнопки управления, строку поиска и выпадающий список для фильтрации по статусу.

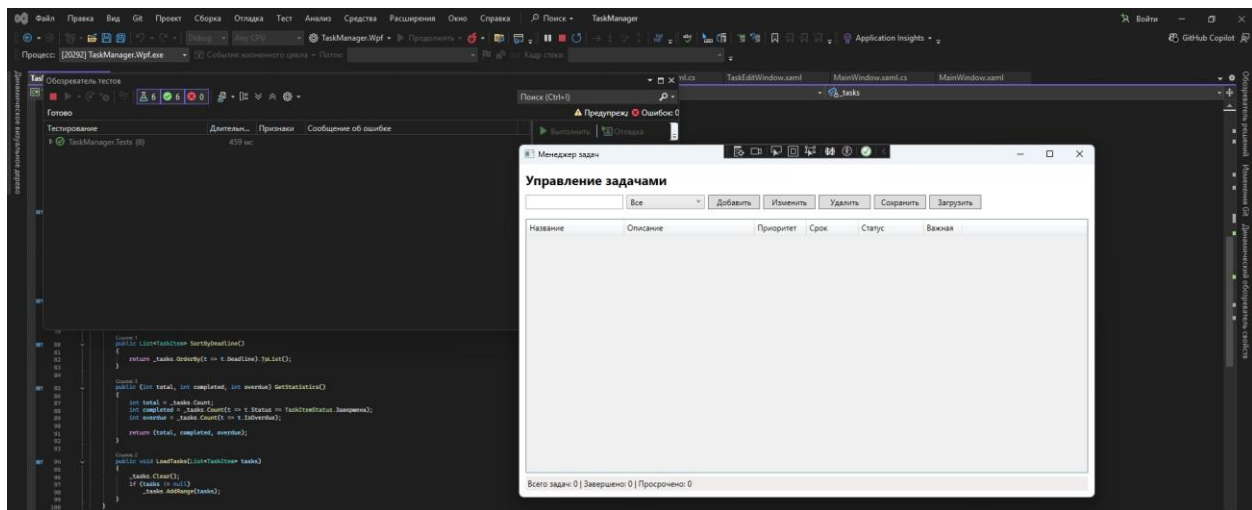


Рисунок 6 — Главное окно приложения

Для добавления и редактирования задач используется отдельное окно:

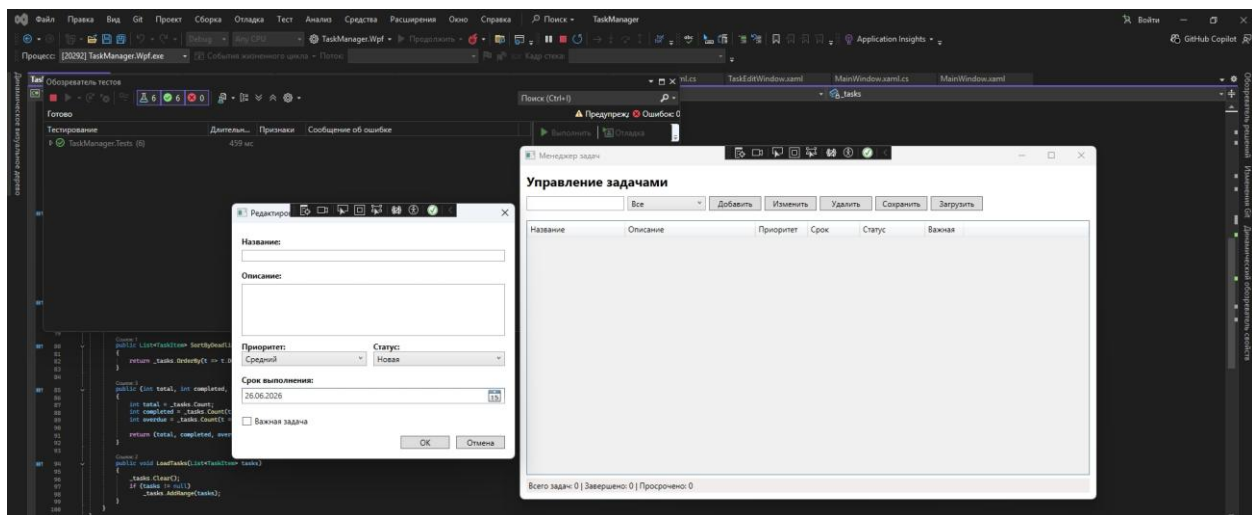


Рисунок 7 — Окно редактирования задачи

4. ТЕСТИРОВАНИЕ

Для проверки корректности работы библиотеки классов созданы модульные тесты на базе xUnit. Всего реализовано 6 тестов:

1. AddTask_ValidTask_ShouldAddToList — проверяет добавление задачи.
2. AddTask_EmptyTitle_ShouldThrowException — проверяет, что нельзя добавить задачу с пустым названием.
3. FilterByStatus_ShouldReturnCorrectTasks — проверяет фильтрацию по статусу.
4. Search_ShouldFindByTitle — проверяет поиск по названию.
5. DeleteTask_ExistingId_ShouldRemoveTask — проверяет удаление задачи.
6. GetStatistics_ShouldCountCorrectly — проверяет корректность подсчёта статистики.

Все тесты успешно пройдены, что подтверждает правильность работы бизнес-логики приложения.

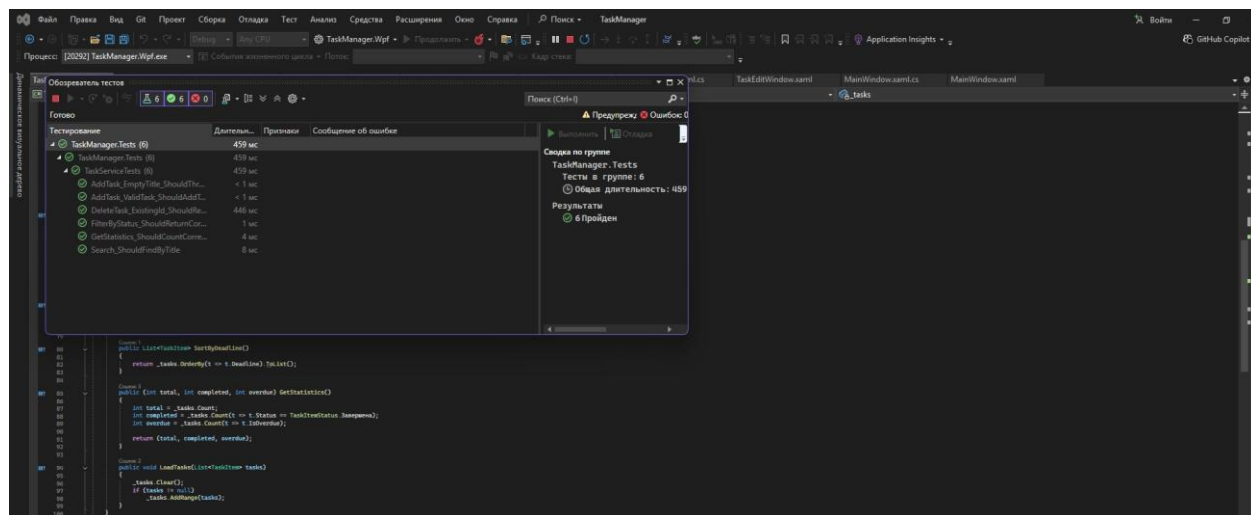


Рисунок 8 — Результаты выполнения модульных тестов

5. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Добавление задачи:

1. Нажать кнопку «Добавить».
2. Заполнить поля: название, описание, выбрать приоритет и статус.
3. Указать срок выполнения.
4. При необходимости отметить «Важная задача».
5. Нажать «ОК».

Редактирование задачи:

1. Выбрать задачу в таблице.
2. Нажать кнопку «Изменить».
3. Внести изменения в поля.
4. Нажать «ОК».

Удаление задачи:

1. Выбрать задачу в таблице.
2. Нажать кнопку «Удалить».
3. Подтвердить удаление.

Поиск:

Ввести текст в поле поиска — таблица обновится автоматически, отображая задачи, содержащие введённый текст в названии или описании.

Фильтрация по статусу:

Выбрать нужный статус из выпадающего списка. Для отображения всех задач выбрать «Все».

Сохранение в файл:

Нажать кнопку «Сохранить», выбрать папку и ввести имя файла.
Задачи сохраняются в формате JSON.

Загрузка из файла:

Нажать кнопку «Загрузить», выбрать ранее сохранённый файл JSON.
Список задач будет восстановлен.

Статистика:

В нижней части главного окна отображается общее количество задач, количество завершённых и количество просроченных задач.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы было разработано приложение «Менеджер задач» на языке C#. Реализованы все требуемые функции: добавление, редактирование, удаление, фильтрация, поиск, сортировка задач, сохранение и загрузка в JSON, а также модульные тесты.

Архитектура приложения разделена на три проекта:

- библиотека классов (бизнес-логика),
- WPF-приложение (пользовательский интерфейс),
- проект модульных тестов (проверка логики).

Такое разделение соответствует принципам ООП и позволяет легко модифицировать или расширять приложение в будущем.

В процессе работы получены и закреплены практические навыки:

- работы с классами, интерфейсами и перечислениями в C#;
- использования коллекций List<T> и LINQ-запросов;
- файлового ввода-вывода в формате JSON;
- создания графического интерфейса на WPF;
- написания и запуска модульных тестов на xUnit;
- работы в среде Visual Studio и системы контроля версий Git.

Все поставленные задачи выполнены в полном объёме.

СПИСОК ЛИТЕРАТУРЫ

1. Троелсен Э., Джепикс Ф. Язык программирования C# 10 и платформа .NET 6. — М.: Вильямс, 2023.
2. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5. — СПб.: Питер, 2020.
3. Документация Microsoft: WPF. —
[Электронный ресурс] URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/>
4. Документация Microsoft: LINQ. —
[Электронный ресурс] URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/linq/>
5. Документация Microsoft: xUnit. —
[Электронный ресурс] URL: <https://xunit.net/>